

REINFORCEMENT LEARNING: APRENDER A CONDUCIR EXPLORANDO

Caso Práctico 5 · Dossier del alumno

Máster en IA — AIC

Módulo: IA Aplicada al Vehículo Autónomo · 2026

Alcance y requisitos previos

En este quinto y último caso práctico entrenará un agente de **Reinforcement Learning (RL)** que aprende a conducir **explorando por ensayo y error** — no imitando a un experto [eso fue CP4]. Vivirá los puntos dolorosos clásicos del RL: la **ineficiencia de datos**, y el **reward hacking** [cuando el agente "hace trampa" con la recompensa].

Está pensado para realizarse **sin conocimientos previos de programación**. El ordenador ya se preparó en CP1; aquí se reutiliza [con un entorno propio de CP5].



Duración estimada: ~50–65 min. Sin GPU. 8 GB de RAM. El entrenamiento del agente tarda ~8–10 min.

Objeto de la práctica

Es el **complemento de CP4**: mismo simulador (`highway-env`), **paradigma opuesto**. En vez de imitar, el agente **prueba acciones y aprende de la recompensa**.

1. **Explorar** el entorno (observación, acciones, recompensa).
2. **Baselines**: un agente **aleatorio** y uno **por reglas** simples.
3. **Entrenar un DQN** (Deep Q-Network) con `stable-baselines3`.
4. **Evaluar** sobre 50 episodios y comparar con los baselines.
5. **Reward shaping**: cambiar la recompensa y descubrir el **reward hacking**.

La pregunta del caso: ¿por qué la industria casi nunca despliega RL puro en un coche?

Objetivos de aprendizaje

Al finalizar la práctica, el alumno será capaz de:

1. Entender la diferencia entre **imitar** (CP4) y **explorar** (RL).
2. Entrenar un agente **DQN** y leer su **curva de aprendizaje**.
3. Comparar RL frente a baselines simples (aleatorio, reglas).
4. Reconocer la **ineficiencia de datos** del RL (cuánta "experiencia" necesita).
5. Provocar y detectar **reward hacking**: cómo una recompensa mal diseñada produce un comportamiento indeseado.

Estructura del trabajo

Archivo	Función	Duración
02_setup	Verifica el entorno + SB3	5 min
03_entorno	Explora observación, acciones, recompensa	8 min
04_baselines	Agente aleatorio + por reglas	8 min
05_dqn_entrenamiento	Entrena el DQN (~8-10 min)	14 min
06_evaluacion	Evalúa (50 episodios) + GIFs	8 min
07_reward_shaping	Cambia la recompensa → reward hacking	10 min
08_preguntas	Cuestionario de entrega	10 min

PARTE 0

Preparación

Lo que hace falta antes de empezar

El ordenador ya quedó preparado en CP1. Para CP5:

1. **Tener la carpeta** `CP5-highway-rl` (mismo repositorio: **Code** → **Download ZIP** si no la tiene).
2. Crear un **entorno virtual** propio de CP5 e instalar dependencias.

Repositorio: <https://github.com/codx-digital/master-ia-practicas>

No se descarga dataset ni modelo: **todo se genera y entrena en local.**

Paso 1 · Entorno de CP5 + dependencias

Entre en la carpeta `CP5-highway-rl` y:

```
python -m venv .venv
```

```
.venv\Scripts\activate    ← Windows
source .venv/bin/activate ← Mac
```



Debe aparecer `(.venv)`.

```
pip install --upgrade pip
pip install -r requirements.txt
```

Instala el simulador [`highway-env`], la librería de RL [`stable-baselines3`] y PyTorch. Tarda unos minutos.

Paso 2 · Abrir Jupyter y verificar [02_setup]

```
jupyter notebook
```

Ejecute `02_setup.ipynb` entero. Comprueba que el simulador y `stable-baselines3` están bien (incluye un mini-entrenamiento de prueba). Debe terminar con  `Setup OK`.



Si ve avisos tipo `objc[...] SDL ... implemented in both`, **ignórelos** — son inofensivos.

PARTE 1 • EL ENTORNO Y LOS BASELINES

Notebooks 03_entorno y 04_baselines • ~16 min

Qué hacen estos notebooks

03_entorno te enseña las **tres piezas** del RL en este simulador:

- **Observación:** qué ve el agente (posición y velocidad de los coches).
- **Acciones:** qué puede hacer (acelerar, frenar, cambiar de carril...).
- **Recompensa:** qué premia el entorno (ir rápido sin chocar).

04_baselines ejecuta **dos agentes tontos** para tener con qué comparar:

- **Aleatorio:** elige acciones al azar.
- **Por reglas:** una heurística simple.

👉 **Anote** el *return* (recompensa acumulada) medio de cada baseline. Es la vara de medir del DQN.

PARTE 2 • ENTRENAR EL AGENTE

Notebook 05_dqn_entrenamiento • ~12 min • el corazón

Qué hace este notebook

Entrena un **DQN** (Deep Q-Network): un agente que aprende, **probando acciones y viendo la recompensa**, qué conviene hacer en cada situación. En **CPU**, ~8–10 minutos, 40.000 pasos de experiencia.



Observa la curva de aprendizaje: al principio el agente es malo (return bajo, como el aleatorio); con el tiempo **sube** a medida que aprende. Ese ascenso es el aprendizaje por refuerzo en directo.



Anote en qué momento (timestep) el DQN empieza a **superar al baseline por reglas** — lo necesitas para la Pregunta 2 [eficiencia de datos].



El entrenamiento tarda; es normal que la barra de progreso avance despacio. No lo pares.

PARTE 3 • EVALUACIÓN

Notebook 06_evaluacion • ~8 min

Qué hace este notebook

Evalúa el DQN entrenado sobre **50 episodios nuevos** (semillas distintas a las de entrenamiento) y lo **compara** con los baselines. Genera además **GIFs** de algunos episodios para verlo conducir.

Casi siempre el DQN **supera** al aleatorio, y con suerte al de reglas. Si **no** lo supera, no pasa nada: es material para la Pregunta 1 (¿por qué? ¿pocos timesteps?).

📝 **Anote** los tres returns (aleatorio, reglas, DQN) — es la Pregunta 1.

PARTE 4 • REWARD HACKING

Notebook `07_reward_shaping` • ~10 min • la lección clave

💡 Cuidado con lo que premias

Aquí cambias la recompensa: le añades una **penalización por cambiar de carril** (para que conduzca "más tranquilo") y reentrenas un poco.



Lo que vas a descubrir: el agente, para evitar la penalización, **deja de cambiar de carril...** pero puede acabar **chocando más** (no esquivar). Ha aprendido a "ganar puntos" de una forma que **no era la intención**. Eso es el **reward hacking**.

Es la lección central del RL: el agente optimiza **exactamente lo que le pides**, no lo que **quieres**. Diseñar bien la recompensa es difícilísimo, y por eso el RL puro es peligroso en un coche.



Observa: ¿bajaron los cambios de carril? ¿subió la tasa de colisión? (Pregunta 3.)

PARTE 5 • EL ENTREGABLE

Documento a subir a Moodle

Qué se entrega

Un **PDF** (`cp5_<apellido>_<nombre>.pdf`), a **Moodle** (sección "CP5 — RL"), en **48 h**. Use la **plantilla** `plantilla-respuestas-cp5.docx` [carpeta `entregable/`].

Responde a **5 preguntas** apoyándote en tus resultados:

1. **Random vs Reglas vs DQN** — los 3 returns; ¿cuál gana y por qué?
2. **Eficiencia de datos** — ¿cuánta experiencia necesitó el DQN? Compáralo con el BC de CP4.
3. **Reward hacking** — ¿qué pasó al penalizar el cambio de carril? ¿Cómo lo arreglarías?
4. **Simulador vs mundo real** — 3 simplificaciones de highway-env.
5. **Sim-to-real** — ¿qué pasos darías para llevar esto a un coche real?

Cómo se te evalúa (16 puntos)

Se valora	En qué consiste
Ejecución	Entrenas el DQN, evalúas y haces el reward shaping
Comprensión	Entiendes RL vs baselines y la eficiencia de datos
Análisis crítico	Detectas el reward hacking y propones mitigación
Comunicación	PDF ordenado, con la curva de aprendizaje y la comparativa

Lo que **más sube la nota**: números concretos (returns, timesteps), y un análisis honesto del reward hacking + una mitigación específica.

Normas de la actividad

- Se **permite** el uso de herramientas de IA como apoyo, pero el análisis y las conclusiones deben ser **propios**. Copiar penaliza.
- Si usó una de estas herramientas, **decláralo** en una línea. No baja la nota; ocultarlo, sí.
- La carpeta **soluciones/** **no debe consultarse** hasta haber intentado el caso.

PARA COMENZAR

Orden: 02 → 03 → 04 → 05 → 06 → 07 → responder 08 .

El objetivo no es un agente perfecto — es **entender por qué el RL es poderoso pero difícil de controlar**. Ante dudas, el **foro del máster**.

"En Behavioral Cloning el problema era imitar. En RL el problema es **decirle a la máquina qué quieres** sin que haga trampa. Diseñar la recompensa correcta es medio arte, medio ingeniería."